CompQsoft Marketing & Webinar Automation

Build plan for a single dashboard that replaces the Constant Contact + Microsoft Teams + Excel juggling.

TypeScript backend | Constant Contact API | Microsoft Graph (Teams) API

Prepared for: InvisiEdge → CompQsoft account • Date: 10 July 2026 • Status: draft for review

In one line

Today the webinar-marketing process is spread across five tools and a lot of manual Excel work. This plan builds **one dashboard** where the team manages contacts and segments, creates and schedules email campaigns, spins up a Teams webinar in one click, captures registrations on CompQsoft's own website, and gets attendance and account-planning reports automatically. Constant Contact and Teams stay as the engines underneath (through their APIs); the team only ever opens our dashboard.

1. Context and goal

From the meeting, the current process looks like this:

- **Prospect data** is pulled from LinkedIn Sales Navigator, enriched in LeadGen (email + phone), and exported to Excel.
- **Constant Contact** holds the contacts, splits them into segments (by industry, by Microsoft AE, and by persona), sends the webinar email campaigns, and shows opens / clicks.
- **Microsoft Teams** actually hosts each webinar, handles registration, and sends the reminder emails.
- **The website (Wix)** sits on the side and gets almost no traffic (about 600 visitors in two years).
- **Excel, by hand**, is used to reconcile who attended and to build the account-planning report that goes to the sales team (Greg, John, Amy).

One person (Sarah) is stitching all of this together manually, downloading attendance from Teams into Excel and hand-building reports. The goal Santhosh set is clear: **stop juggling platforms, do everything from one dashboard, and drive traffic to CompQsoft's own website.** Two concrete targets came out of the call:

- **Consolidate** the scattered flow into one-click actions with a single dashboard and automatic reports (no more manual Excel).
- **Own the traffic and the data** by hosting event / registration pages on CompQsoft's own site instead of Teams or Constant Contact, which also helps SEO and keeps registrant data in our own database.

2. What the dashboard replaces

Every manual step below maps to an automated feature. This is the heart of the build.

Step in today's process	Tool now	In the new dashboard
Gather prospect / AE data	Sales Navigator, LeadGen, Excel	Lead import (CSV or LeadGen), auto-tagged by industry and persona
Add contacts and segment them	Constant Contact (manual)	Contacts + Segments module; one-click sync into Constant Contact
Get the email template	A separate person	Template picker; the campaign is built from a saved template
Send weekly campaigns (4-5 'volumes')	Constant Contact	Campaign scheduler queues all volumes at once via the CC API
Create the webinar + registration	Microsoft Teams (manual)	One-click 'Create webinar' via the Teams Graph API (draft → publish)

Step in today's process	Tool now	In the new dashboard
Host the registration page	Teams / Constant Contact page	Registration page on CompQsoft's own website; data saved to our DB
Track opens and clicks	Constant Contact dashboard	Analytics synced automatically, per contact and per account
Get the attendee list	Teams → manual Excel	Attendance synced automatically (with time attended)
Remove bounced emails	Constant Contact (manual)	Hard bounces removed automatically after each send
Build the account plan	Excel, by hand	Account plan generated automatically; one-click export

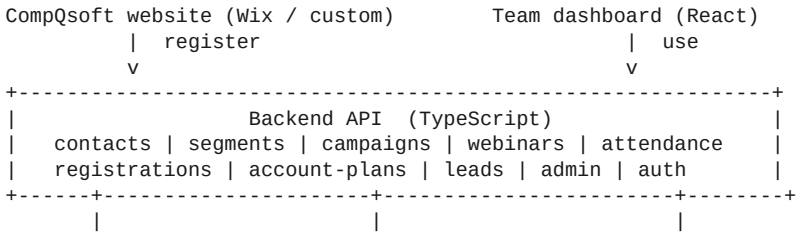
3. Feature modules

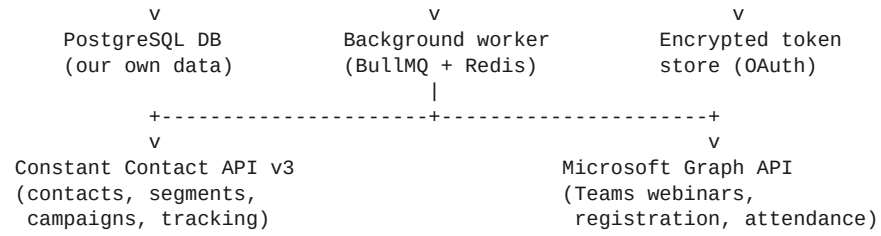
- **Contacts and segmentation** — add / edit contacts, custom fields for industry and persona (IT, Line of Business, Customer Service), build segments, push them to Constant Contact.
- **Email campaigns** — create from a template, preview / test-send, schedule the weekly volumes, and read back the results.
- **Webinars (Teams)** — create and publish a Teams webinar in one click, get the join link, keep Teams' own attendee emails switched off so we control the messaging.
- **Registration** — a registration form on CompQsoft's site that saves the registrant to our database and registers them on Teams behind the scenes.
- **Attendance and reporting** — automatic attendance pull after each webinar, matched back to contacts and registrations.
- **Account planning** — per-account view of people by persona, emails sent, opens, unique clicks, who clicked, who attended and for how long; exportable to Excel / PDF for the sales team.
- **Lead import** — bring lead lists in from CSV or LeadGen into contacts and segments.
- **Admin** — dashboard users, roles, per-user activity, and the two API connections (Constant Contact and Microsoft).

4. High-level architecture

Five moving parts. The backend is the focus of this plan.

- **Web dashboard (frontend)** — what the team uses. React / Next.js. Out of scope here, but it talks only to our backend.
- **Backend API (TypeScript / Node)** — the brain. Holds all logic and talks to Constant Contact and Microsoft on our behalf.
- **PostgreSQL database** — our own copy of contacts, campaigns, webinars, registrations, attendance and stats.
- **Background worker** — a job queue that syncs campaign stats and attendance, fires scheduled sends, and refreshes API tokens.
- **Public registration endpoint** — a small public route the website's event pages call when someone registers.





5. Tech stack

- **Runtime:** Node.js 20 LTS.
- **Language:** TypeScript (strict mode).
- **Framework:** NestJS — recommended because it keeps each integration in its own tidy module. (Plain Express + TypeScript works too if you want something lighter.)
- **Database + ORM:** PostgreSQL with Prisma (fully typed, migrations built in).
- **Jobs / queue:** BullMQ on Redis for scheduled syncs and campaign sends; repeatable jobs for polling.
- **HTTP:** axios (or native fetch) wrapped in a small typed client per API, with automatic token refresh and retry / backoff.
- **Validation:** zod (or class-validator) on every request body.
- **Auth (dashboard users):** JWT + role-based access (admin vs member).
- **Secrets:** environment variables; OAuth tokens encrypted at rest (AES-256). Nothing hard-coded.
- **Hosting:** any Node host. Azure App Service is a natural fit given the Microsoft side, but Render / Railway / AWS are all fine.

6. Data model (PostgreSQL)

Core tables. Every row that mirrors an external record keeps that external ID so syncing stays idempotent (we update, never duplicate).

- **users** — dashboard login, role
- **oauth_connections** — provider, access_token, refresh_token, expires_at, scope (encrypted)
- **organizations** — company name, industry, revenue, AE owner
- **contacts** — name, email, title, org_id, industry, persona, source, cc_contact_id
- **segments** — name, type (industry / AE / persona), criteria; + segment_members join table
- **campaigns** — name, subject, template_id, webinar_id, volume_number, status, scheduled_at, cc_campaign_id, cc_activity_id
- **campaign_stats** — campaign_id, sends, opens, unique_opens, clicks, unique_clicks, bounces, optouts, last_synced_at
- **contact_campaign_activity** — contact_id, campaign_id, opened, clicked, open_count, click_count, first_open_at (this powers account planning)
- **webinars** — title, description, start, end, organizer_upn, status, ms_webinar_id, ms_session_id, join_url, registration_url
- **registrations** — webinar_id, contact/email, source (website / teams), registered_at, ms_registration_id, join_url
- **attendance** — webinar_id, contact/email, join_time, leave_time, duration_seconds, attended, ms_attendance_record_id
- **lead_import_jobs** — source, status, count, file_ref
- **audit_log** — user, action, entity, timestamp

7. The two integrations

7a. Constant Contact API (v3)

Base URL `https://api.cc.email/v3`. Uses OAuth2 (authorization-code). We store the refresh token once and auto-refresh the short-lived access token before every batch of calls.

What we call and what it gives us:

- **Contacts** — create / update / delete, with custom fields for industry and persona.
- **Bulk import** (`/activities/contacts_json_import`) — add whole lead lists in one go.
- **Contact lists + Segments** — build the industry / AE / persona segments.
- **Email campaigns** — create the campaign and its activity, preview / test-send, and schedule sends (`/emails/activities/{id}/schedules`).
- **Tracking reports** — `/reports/email_reports/{campaign_activity_id}/tracking/{opens|clicks|bounces|optouts}`. These return **per-contact** data: who opened, who clicked which link, when, and on what device. That is exactly the account-planning input, plus the bounce list we use to auto-clean the segment and protect domain reputation.

Illustrative client (token refresh built in):

```
// integrations/constant-contact/cc.client.ts
import axios, { AxiosInstance } from "axios";
import { getConnection, saveTokens } from "../token-store";

const BASE = "https://api.cc.email/v3";

export class ConstantContactClient {
  private http: AxiosInstance;

  constructor(private accountId: string) {
    this.http = axios.create({ baseURL: BASE });
    this.http.interceptors.request.use(async (cfg) => {
      cfg.headers.Authorization = `Bearer ${await this.token()}`;
      return cfg;
    });
  }

  private async token(): Promise<string> {
    const c = await getConnection(this.accountId, "constant_contact");
    if (c.expiresAt > Date.now() + 60_000) return c.accessToken;
    return this.refresh(c.refreshToken);      // POST oauth token, saveTokens()
  }

  getClicks(activityId: string) {
    return this.http.get(
      `/reports/email_reports/${activityId}/tracking/clicks`);
  }
  // importContacts(), createCampaign(), scheduleSend(), getBounces() ...
}
```

7b. Microsoft Teams (Graph API)

This is the trickier half, so read the setup carefully — a couple of things are easy to miss and cause 403 errors.

- **App registration:** register an app in Azure AD (Microsoft Entra).
- **Two permission modes** (Teams enforces different rules per operation):
  - **Create / publish a webinar** and register internal users: **delegated** `VirtualEvent.ReadWrite`, acting on behalf of the organizer account (the `webinar@...digital.com` mailbox Sarah mentioned). Create as a draft, then publish.
  - **List webinars, read attendance, register external (anonymous) people: application** permissions — `VirtualEvent.Read.All`, `VirtualEventRegistration-Anon.ReadWrite.All`, `OnlineMeetingArtifact.Read.All`.

- **Application Access Policy (the #1 gotcha):** for app-only access to a user's webinars / attendance, a Teams admin must create and grant an Application Access Policy for the organizer account in Teams PowerShell (`New-CsApplicationAccessPolicy` then `Grant-CsApplicationAccessPolicy`). Without it, webinar and attendance calls return 403 even with the right permissions.
- **Licensing:** the organizer needs Microsoft 365 E3 / E5 or Teams Premium to create a webinar through Graph.

What we call and what it gives us:

- **Create webinar** — `POST /solutions/virtualEvents/webinars` (draft) then `.../publish`. Set `isAttendeeEmailNotificationEnabled = false` so Teams stays quiet and we send comms from our own domain and site.
- **Get the join link** — list sessions and read `joinWebUrl`.
- **Register a person** — `POST /webinars/{id}/registrations` returns that registrant's unique join URL.
- **Attendance** — after the webinar ends Teams generates a report; we read `.../sessions/{sid}/attendanceReports/{rid}?$expand=attendanceRecords` to get each attendee's email, join / leave time, and total seconds attended. This is the manual Excel step, gone.
- **Optional webhooks** — subscribe to virtual-event change notifications so attendance and registration update by push instead of polling.

Illustrative webinar-create service:

```
// integrations/microsoft-graph/graph.service.ts
async createWebinar(input: NewWebinar): Promise<Webinar> {
  // 1) create as draft  (delegated: VirtualEvent.ReadWrite)
  const draft = await this.graph.post(
    "/solutions/virtualEvents/webinars", {
      displayName: input.title,
      startDateTime: { dateTime: input.start, timeZone: input.tz },
      endDateTime:   { dateTime: input.end,   timeZone: input.tz },
      settings: { isAttendeeEmailNotificationEnabled: false },
    });

  // 2) publish
  await this.graph.post(
    `/solutions/virtualEvents/webinars/${draft.id}/publish`, {});

  // 3) read the session join link
  const sessions = await this.graph.get(
    `/solutions/virtualEvents/webinars/${draft.id}/sessions`);

  return this.toWebinar(draft, sessions.value[0]);   // save to our DB
}
```

8. Backend project structure

A NestJS-style layout. Each external service lives in one folder with a typed client, a service, and its types.

```
src/
  main.ts
  app.module.ts
  config/                     env + settings
  common/                     guards, interceptors, encryption, errors
  auth/                       dashboard login, JWT, roles
  integrations/
    token-store/              encrypted OAuth token persistence
    constant-contact/         cc.client.ts  cc.service.ts  cc.types.ts
    microsoft-graph/          graph.client.ts  graph.service.ts  graph.types.ts
  modules/
    contacts/                 controller + service + dto
    segments/
    campaigns/
    webinars/
    registrations/            includes the PUBLIC registration route
    attendance/
    account-plans/
    leads/                    CSV / LeadGen import
```

```
  admin/
jobs/
  queue.module.ts
  sync-campaign-stats.job.ts
  sync-attendance.job.ts
  schedule-campaign.job.ts
  refresh-tokens.job.ts
prisma/                     schema.prisma + migrations
```

9. Internal API (what the frontend calls)

Auth & connections

- `POST /auth/login`, `GET /auth/me`
- `GET/POST /connections/constant-contact/oauth` (start + callback)
- `GET/POST /connections/microsoft/oauth` (start + callback)

Contacts & segments

- `GET/POST/PUT/DELETE /contacts`, `POST /contacts/import`
- `GET/POST /segments`, `POST /segments/:id/sync-to-cc`

Campaigns

- `GET/POST /campaigns`, `POST /campaigns/:id/schedule`, `POST /campaigns/:id/send-test`, `GET /campaigns/:id/stats`

Webinars & registration

- `GET/POST /webinars`, `POST /webinars/:id/publish`, `GET /webinars/:id/join-url`
- `POST /public/webinars/:slug/register` (called by the website event page)

Attendance & account plans

- `GET /webinars/:id/attendance`, `POST /webinars/:id/attendance/sync`
- `GET /accounts/:id/plan`, `GET /webinars/:id/account-plan`, `GET .../export` (xlsx / pdf)

Admin

- `GET /admin/usage`, users CRUD

10. Key workflows, end to end

A. Create and promote a webinar (one click)

- Team fills the webinar form (title, description, date, target industry).
- Backend → Graph: create draft → set attendee emails off → publish → read the join link. Save it all to our DB.
- Backend exposes / creates the registration page on CompQsoft's website.
- Team picks the target segment; backend makes sure those contacts exist in Constant Contact, builds the campaign from a template, and schedules the weekly volumes (4-5 sends).
- **Result:** webinar live, registration on our own site, campaigns queued — from one screen.

B. Someone registers on our website

- Visitor submits the form on CompQsoft's site.
- The public route saves them to our DB, then Backend → Graph registers them on Teams and gets their unique join URL.
- We send the confirmation from our own domain.
- **Result:** registrant data lives in our DB (good for SEO and follow-up) and in Teams.

C. Campaign analytics sync

- A scheduled job runs after each send and periodically.
- Backend → Constant Contact: pull opens / clicks / bounces per campaign and upsert into `campaign_stats` and `contact_campaign_activity`.
- Hard bounces are removed from the list automatically.
- **Result:** live per-person open / click data with no manual export.

D. Attendance sync (replaces the manual Excel)

- When the webinar ends, Teams generates the attendance report.
- A webhook (or hourly job) triggers Backend → Graph to fetch the report and its records, then upsert attendance rows (email, join / leave, seconds).
- Attendees are matched to registrations and contacts.
- **Result:** automatic attendance with duration — no Excel reconciliation.

E. Account-plan generation

- For an org (or a webinar) the backend aggregates: people by persona (IT / LOB / CS), emails sent, opens, unique clicks, who clicked, who attended and for how long.
- Shown on the dashboard and exportable to Excel / PDF for Greg, John, Amy.
- **Result:** the report Sarah builds by hand, generated instantly.

Illustrative attendance-sync job:

```
// jobs/sync-attendance.job.ts
export async function syncAttendance(webinarId: string) {
  const w = await db.webinar.findByIdOrThrow(webinarId);

  const res = await graph.get(
    `/solutions/virtualEvents/webinars/${w.msWebinarId}` +
    `/sessions/${w.msSessionId}` +
    `/attendanceReports?$expand=attendanceRecords`);

  for (const rec of res.value[0]?.attendanceRecords ?? []) {
    await db.attendance.upsert({
      webinarId,
      email: rec.emailAddress,
      durationSeconds: rec.totalAttendanceInSeconds,
      joinTime: rec.attendanceIntervals?.[0]?.joinDateTime,
      attended: true,
    });
  }
}
```

11. Background jobs and sync strategy

- **syncCampaignStats** — every few minutes for active campaigns.
- **syncAttendance** — triggered after a webinar ends, with an hourly fallback.
- **scheduleCampaign** — fires the scheduled volume sends.
- **refreshTokens** — refresh OAuth access tokens before they expire.
- Prefer Graph **change-notification webhooks** for attendance and registration to cut down polling.
- All writes are **idempotent upserts** keyed on external IDs (cc_contact_id, ms_registration_id, ms_attendance_record_id) so re-runs never duplicate data.

12. Auth and security

- Store Constant Contact and Microsoft OAuth tokens **encrypted** in `oauth_connections` — never in code.
- Auto-refresh tokens; on a 401, refresh once and retry.

- Dashboard uses JWT + roles (admin vs member), which also gives you the per-user view and admin usage screen (like the LeadGen credit model the team already uses).
- Handle rate limits on both APIs with retry / backoff.
- Restrict access to PII (emails, phones) and record access in `audit_log`.

13. Setup checklist (before writing feature code)

- **Constant Contact:** developer account → create an app → get client id / secret → request scopes (contact + campaign + account) → complete OAuth once to obtain the refresh token.
- **Azure / Microsoft:** app registration → add Graph permissions (delegated `VirtualEvent.ReadWrite`; application `VirtualEvent.Read.All`, `VirtualEventRegistration-Anon.ReadWrite.All`, `OnlineMeetingArtifact.Read.All`) → grant admin consent → add a client secret or certificate.
- **Teams admin:** create and grant an Application Access Policy for the organizer account (the webinar mailbox). Skip this and webinar / attendance calls will 403.
- **Licensing:** confirm the organizer has M365 E3 / E5 or Teams Premium.
- **Website:** agree with Lokesh / Sohit how the event pages call our registration route (custom code, iframe, or Wix HTTP functions) — or whether to move to a custom site we fully control.
- **Credentials:** collect the webinar organizer login from Sarah, as agreed in the call.

14. Phased roadmap

Phase 0 — Foundations. Repo, database schema, the two OAuth connections, and the encrypted token store.

Phase 1 — MVP (the Constant Contact side, UI-first). Contacts + segments (with CC sync), create and schedule an email campaign, the analytics dashboard, and the account-plan view + export. This alone pulls the whole Constant Contact workflow onto one screen — enough to demo to Greg. Santhosh wanted a UI prototype for the Monday call, so this is the target for that demo (mock the parts that aren't wired up yet).

Phase 2 — Teams webinars. One-click create + publish, registration capture on the website pushed into Teams, and automatic attendance sync.

Phase 3 — Lead pipeline + polish. CSV / LeadGen import, webhooks instead of polling, admin usage dashboard, audit log, and a template library.

Phase 4 — Optional handoffs. Export to Stephen for cold calling and an optional SalesHandy sequence integration.

15. Open decisions

- **Website platform:** stay on Wix, or move to a custom site we control? A custom site gives the cleanest registration + database integration and the best SEO, which is the whole point of driving traffic there.
- **Email ownership:** recommend turning Teams' attendee emails off and sending everything from our own domain and site.
- **Templates:** reuse Constant Contact templates, or build our own template editor in the dashboard?
- **Organizers:** one shared webinar organizer account, or several? (Affects the scope of the Application Access Policy.)
- **Data residency** for stored PII.

End of plan. This is a starting blueprint — exact endpoint paths, scopes and permissions should be confirmed against the current Constant Contact v3 and Microsoft Graph docs during setup, since both evolve.